Amruta Institute of Engineering and Management Sciences – Polytechnic

Subject Code: 20CS41P

Subject: Data Structures with Python

Week 1

1. Python program to Use and demonstrate basic data structures.

Write a python program to read student name and marks of 4 subjects, calculate total and average marks of 4 subjects and print the details with a data type.

```
print("Python program to Use and demonstrate basic data structures\n")
name=input("Enter name:")
python=int(input("Enter python marks:"))
os=int(input("Enter os marks:"))
java=int(input("Enter java marks:"))
se=int(input("Enter se marks:"))
total=python+os+java+se
avg=total/4
print("\nStudent name is:", name,"Data type is:",type(name))
print("python marks is:", python,"Data type is:",type(python))
print("os marks is:", os,"Data type is:",type(os))
print("java marks is:", java,"Data type is:",type(java))
print("se marks is:", se,"Data type is:",type(se))
print("\ntotal marks is:", total,"Data type is:",type(total))
print("\nAverage is:", avg,"Data type is:",type(avg))
if avg>90:
 a=True
else:
 a=False
 print("\nAverage is greater than 90 ?", a, "Data type is:", type(a))
 print("\nList is a collection which is ordered and changeable. Allows duplicate members")
```

```
example = ["bgsp", "cse", "4"]
print(example, "Data type is:", type(example))
print("\nTuple is a collection which is ordered and unchangeable. Allows duplicate members.")
example = ("bgsp", "cse", "4", "cse")
print(example, "Data type is:", type(example))
print("\nSet is a collection which is unordered, unchangeable, and unindexed. No duplicate members")
example = {"bgsp", "cse", "4"}
print(example, "Data type is:", type(example))
print("\nDictionary is a collection which is ordered and changeable. No duplicate members")
example = {"College": "bgsp", "branch": "cse", "sem": "4"}
print(example, "Data type is:", type(example))
```

2. Implement an ADT with all its operations.

```
class student:
    def __init__(self):
        self.rollno = input("Enter Roll Number : ")
        self.name = input("Enter Name : ")
        self.python = int(input("Enter Python Marks : "))
        self.java = int(input("Enter JAVA Marks : "))
        self.os = int(input("Enter OS Marks : "))
        self.se = int(input("Enter SE Marks : "))

    def display(self):
        print("\nStudent name is:", self.name)
        print("Student Roll_NO is:", self.rollno)
        print("Python marks is:", self.python)
        print("java marks is:", self.java)
        print("OS marks is:", self.os)
        print("SE marks is:", self.se)

    def total_avg(self):
        total = self.python + self.java + self.os + self.se
        avg = total / 4
```

```
        print("Total marks is:", total)
        print("avg marks is:", avg)

    def result(self):
        if (self.python < 35):
            print("failed in python")
        else:
            print("pass in python")
        if (self.java < 35):
            print("failed in java")
        else:
            print("pass in java")
        if (self.os < 35):
            print("failed in os")
        else:
            print("pass in java")
        if (self.se < 35):
            print("failed in se")
        else:
            print("pass in se")

n = int(input("Enter number of students: "))
students = []

for i in range(n):
    print("\nEnter details of student", i + 1)
    students.append(student())

for s in students:
    s.display()
    s.total_avg()
    s.result()
```

Week 2

1. Implement an ADT and Compute space and time complexities.

Write a program to

i) create student class with constructor that reads student register number, name, marks of 4 subjects

ii) display register number, name, marks of 4 subjects

iii) find the result of each subject

```
class student:
    def __init__(self):
        self.rollno = input("Enter Roll Number : ")
        self.name = input("Enter Name : ")
        self.python = int(input("Enter Python Marks : "))
        self.java = int(input("Enter JAVA Marks : "))
        self.os = int(input("Enter OS Marks : "))
        self.se = int(input("Enter SE Marks : "))

    def display(self):
        print("\nStdeunt name is:", self.name)
        print("Stdeunt Roll_NO is:", self.rollno)
        print("Python marks is:", self.python)
        print("java marks is:", self.java)
        print("OS marks is:", self.os)
        print("SE marks is:", self.se)

    def total_avg(self):
        total = self.python + self.java + self.os + self.se
        avg = total / 4
        print("Total marks is:", total)
        print("avg marks is:", avg)

    def result(self):
        if (self.python < 35):
            print("failed in python")
```

```
        else:
            print("pass in python")
        if (self.java < 35):
            print("failed in java")
        else:
            print("pass in java")
        if (self.os < 35):
            print("failed in os")
        else:
            print("pass in java")
        if (self.se < 35):
            print("failed in se")
        else:
            print("pass in se")


n = int(input("Enter number of students: "))
students = []
for i in range(n):
    print("\nEnter details of student", i + 1)
    students.append(student())
for s in students:
    s.display()
    s.total_avg()
    s.result()

import matplotlib.pyplot as p
x = list(range(101))
y = [i for i in x]
p.plot(x, y)
p.title("ADT - Time Complexity")
p.xlabel("Input")
p.ylabel("Time")
p.show()
```

2. Implement above solution using array and Compute space and time complexities and compare two solutions.

Write a program to

i) create student array that reads student register number, name, marks of 4 subjects

ii) display register number, name, marks of 4 subjects

iii) find the result of each subject

```
rollno = []
name = []
python = []
java = []
os = []
se = []
n = int(input("enter number of students"))
for i in range(n):
    rollno.append(input("Enter Roll Number : "))
    name.append(input("Enter Name : "))
    python.append(int(input("Enter Python Marks : ")))
    java.append(int(input("Enter JAVA Marks : ")))
    os.append(int(input("Enter OS Marks : ")))
    se.append(int(input("Enter SE Marks : ")))

for i in range(n):
    print("\nStdeunt name is:", name[i])
    print("Stdeunt Roll_NO is:", rollno[i])
    print("Python marks is:", python[i])
    print("java marks is:", java[i])
    print("OS marks is:", os[i])
    print("SE marks is:", se[i])


    total = python[i] + java[i] + os[i] + se[i]
    avg = total / 4
    print("Total marks is:", total)
    print("avg marks is:", avg)
```

```
    if (python[i] < 35):
        print("failed in python")
    else:
        print("pass in python")
    if (java[i] < 35):
        print("failed in java")
    else:
        print("pass in java")
    if (os[i] < 35):
        print("failed in os")
    else:
        print("pass in java")
    if (se[i] < 35):
        print("failed in se")
    else:
        print("pass in se")
```

```
import matplotlib.pyplot as p
x=list(range(101))
y=[i for i in x]
p.plot(x,y)
p.title("Array - Time Complexity ")
p.xlabel("Input")
p.ylabel("Time")
p.show()
```

Week 3

1. Implement Linear Search compute space and time complexities, plot graph using asymptomatic notations.

```
import time
a = []
def linear_search(a, n, key):
    for i in range(0, n):
        if key == a[i]:
            print("Key element found at position:", i)
            return
    print("Key element not found")
n = int(input("Enter number of elements: "))
for i in range(0, n):
    element = int(input("Enter an element: "))
    a.append(element)
key = int(input("Enter key value: "))
print("An array is:", a)
start = time.time()
print("The key element is:", key)
linear_search(a, n, key)
end = time.time()
print("Time taken to search element using Linear Search:", end - start)
import matplotlib.pyplot as p
x=list(range(101))
y=[i for i in x]
p.plot(x,y)
p.title("Linear search - Time Complexity ")
p.xlabel("Input")
p.ylabel("Time")
p.show()
```

2. Implement Bubble, Selection, insertion sorting algorithms compute space and time complexities, plot graph using asymptomatic notations.

Bubble Sort

```
import time
a = []
def bubble_sort(a, n):
    for i in range(n):
        for j in range(0, n - 1 - i):
            if a[j] > a[j + 1]:
                temp = a[j]
                a[j] = a[j + 1]
                a[j + 1] = temp
    return a

n = int(input("Enter size: "))
for i in range(n):
    value = int(input("Enter values: "))
    a.append(value)

print("Array before sort:", a)
start = time.time()
bubble_sort(a, n)
end = time.time()
print("Array after sort:", a)
print("Time taken to sort array using bubble sort:", end - start)

x = list(range(0, 101))
y = [i * i for i in x]

import matplotlib.pyplot as p
p.plot(x, y)
p.title("Time Complexity of Bubble Sort")
p.xlabel("Input Size (n)")
p.ylabel("Time / Operations")
p.show()
```

Selection Sort

```
a = []

def selection_sort(a, n):
    for i in range(0, n - 1):
        min = i
        for j in range(i + 1, n):
            if a[j] < a[min]:
```

```
                min = j
            # swap A[i] and A[min] using temp
            temp = a[i]
            a[i] = a[min]
            a[min] = temp

        return a

    n = int(input("Enter size: "))
    for i in range(n):
        value = int(input("Enter values: "))
        a.append(value)

    print("The array before sort:", a)
    start = time.time()
    selection_sort(a, n)
    end = time.time()
    print("The array after sort:", a)
    print("Time taken to sort an array using selection sort:", end - start)

    x = list(range(0, 101))
    y = [i * i for i in x]

    import matplotlib.pyplot as p
    p.plot(x, y)
    p.title("Time Complexity of Selection Sort")
    p.xlabel("Input Size (n)")
    p.ylabel("Time / Operations")
    p.show()
```

Insertion Sort

```
import time
a = []
def insertion_sort(a, n):
    for i in range(1, n):
        k = a[i]
        j = i - 1

        while j >= 0 and a[j] > k:
            a[j + 1] = a[j]
            j = j - 1

        a[j + 1] = k
```

```
n = int(input("Enter size: "))
for i in range(n):
    value = (int(input("Enter values: ")))
    a.append(value)

print("The array before sort:", a)
start = time.time()
insertion_sort(a, n)
end = time.time()
print("The array after sort:", a)
print("Time taken to sort an array using insertion sort:", end - start)

import matplotlib.pyplot as p
x=list(range(1,101))
y=[i*i for i in x]
p.plot(x,y)
p.title("Time complexity of Insertion Sort")
p.xlabel("Input")
p.ylabel("Time")
p.show()
```

Week – 4

1. **Implement Binary Search using recursion Compute space and time complexities, plot graph using asymptomatic notations and compare two.**

```
import time
a=[]
def binary_search(a, low, high, key):
    if(low<=high):
        mid=(low + high)//2
        if key is a[mid]:
            print("key element found at position:",mid)
            return
        elif key<a[mid]:
            return binary_search(a, low, mid-1, key)
        else:
            return binary_search(a, mid+1, high, key)
    print("key element not found")

n=int(input("enter number of elements:"))
for i in  range(0, n):
    element=int(input("enter element:"))
    a.append(element)
key=int(input("enter key value:"))
start=time.time()
print("the array is:",a)
print("the key element is:",key)
result=binary_search(a,0,n-1,key)
end=time.time()
print("Time taken to Search an element using Binary search :" ,end-start)

import math
import matplotlib.pyplot as p

x=list(range(1,101))
y=[math.log(i) for i in x]
p.plot(x,y)
p.title("Binary search Time Complexity")
p.xlabel("Input")
p.ylabel("Time")
p.show()
```

2. Implement Merge and quick sorting algorithms compute space and time complexities, plot graph using asymptomatic notations and compare all solutions.

Merge sort

```
import time
a=[]

def divide(a,low,high):
    if(low<high):
        mid=(low+high)//2
        divide(a,low,mid)
        divide(a,mid+1,high)
        merge(a,low,mid,high)

def merge(a,low,mid,high):
        temp=[]
        i=low
        k=low
        j=mid+1
        while(i<=mid and j<=high):
            if(a[i]<a[j]):
                temp.insert(k,a[i])
                i=i+1
                k=k+1
            else:
                temp.insert(k,a[j])
                j=j+1
                k=k+1
        while(i<=mid):
            temp.insert(k,a[i])
            k=k+1
            i=i+1
        while(j<=high):
            temp.insert(k,a[j])
            k=k+1
            j=j+1
        p=low
        m=0
        while(p<k):
                a[p]=temp[m]
                p=p+1
                m=m+1

n=int(input("Enter size:"))
for i in range(0,n):
    v=int(input("Enter values:"))
```

```
    a.append(v)
start=time.time()
print("the array before sort:",a)
divide(a,0,n-1)
end=time.time()
print("the array after sort:",a)
print("time taken to sort an array using Merge sort",end-start)

import math
import matplotlib.pyplot as p
p.title("Merge Sort ")
p.xlabel("Input")
p.ylabel("Time")
x=list(range(1,101))
y=[i*math.log(i) for i in x]
p.plot(x,y)
p.show()
```

Selection sort

```
import time

a=[]

def qs(a,low,high):
   if(low<high):
      pivot=divide(a,low,high)
      qs(a,low,pivot-1)
      qs(a,pivot+1,high)

def divide(a,low,high):
       pivot=a[low]
       i=low+1
       j=high
       while 1:
         while(i<high)and(pivot>=a[i]):
              i=i+1
         while(pivot<a[j]):
               j=j-1
         if i<j:
              a[i],a[j]=a[j],a[i]
         else:
            a[low],a[j]=a[j],a[low]
            return j

n=int(input("Enter size:"))
```

```
for i in range(0,n):
    value=int(input("Enter values:"))
    a.append(value)
start=time.time()
print("the array before sort:",a)
qs(a,0,n-1)
end=time.time()
print("the array after sort:",a)
print("time taken to sort an array using Quick sort",end-start)

import math
import matplotlib.pyplot as p
p.title("Quick Sort ")
p.xlabel("Input")
p.ylabel("Time")
x=list(range(1,101))
y=[i*math.log(i) for i in x]
p.plot(x,y)
p.show()
```

3. Implement Fibonacci sequence with dynamic programming.

```
import time
a=[]
def fib(n):
    if (n==0):
        return 0
    if (n==1):
        return 1
    else:
        a[n]=fib(n-1)+fib(n-2)
        return a[n]

s=time.time()
n=int(input("enter value for n: "))
for i in range(0,n+1):
      a.insert(i,-1)
for i in range(n+1):
    print(fib(i))
e=time.time()
```

```
print("time take to find fibonacci series",e-s)

import matplotlib.pyplot as p
x=list(range(11))
y=[2**i for i in x]
p.plot(x,y)
p.title("fibonacci series")
p.xlabel("input")
p.ylabel("time taken")
p.show()
```

Week 5

1. **Implement Singly linked list (Traversing the Nodes, searching for a Node, Prepending Nodes, Removing Nodes)**

```
class node:
    def __init__(self, data):
        self.data = data
        self.next = None



class singlylinkedlist:
    def __init__(self):
        self.head = None

    # insert at beginning
    def prepend(self, newnode):
        newnode.next = self.head
        self.head = newnode

    # delete from beginning
    def remove(self):
        if self.head is None:
            print("List is empty")
```

```
            else:
                self.head = self.head.next

        # display list
        def display(self):
            temp = self.head
            while temp is not None:
                print(temp.data, end="-->")
                temp = temp.next
            print("None")

        # search element
        def search(self, key):
            temp = self.head
            while temp is not None and temp.data != key:
                temp = temp.next

            if temp is None:
                print("Key not found")
            else:
                print("Key found")



# main program
s = singlylinkedlist()

n = int(input("Enter number of nodes: "))

for i in range(n):
    val = int(input("Enter value: "))
    s.prepend(node(val))

print("\nLinked List:")
s.display()
```

```
key = int(input("\nEnter element to search: "))
s.search(key)

d = int(input("\nHow many nodes to delete from beginning: "))
for i in range(d):
    s.remove()

print("\nList after deletion:")
s.display()
```

Week 6

1. Implement linked list Iterators.

```
class node:
    def __init__(self, data):
        self.data = data
        self.next = None
class iterator:
    def __init__(self):
        self.head = None

    def iterate(self):
        temp = self.head
        while temp:
            yield temp.data
            temp = temp.next
    def display(self):
        if self.head is None:
            print("List is empty")
            return
        for item in self.iterate():
            print(item, end="-->")
        print("None")
```

```
    def prepend(self, newnode):
        newnode.next = self.head
        self.head = newnode

si = iterator()
while True:
    print("\n1.PREPEND \t 2.DISPLAY \t 3.EXIT\n")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        si.prepend(node(data))

    elif choice == 2:
        si.display()

    elif choice == 3:
        break

    else:
        print("Invalid choice")
```

WEEK 7

a. Implement Doubly linked list

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def display(self):
        if self.head is None:
            print("Linked list is empty")
            return

        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")

    def search(self, key):
        temp = self.head
        while temp and temp.data != key:
            temp = temp.next

        if temp:
            print("Key found")
        else:
            print("Key not found")
```

```
    def append(self, newnode):
        if self.head is None:
            self.head = newnode
            self.tail = newnode
        else:
            self.tail.next = newnode
            newnode.prev = self.tail
            self.tail = newnode

    def remove(self, key):
        temp = self.head

        if temp is None:
            print("Linked list is empty")
            return
        while temp and temp.data != key:
            temp = temp.next

        if temp is None:
            print("Key not found")
            return

        # If deleting head
        if temp == self.head:
            self.head = temp.next
            if self.head:
                self.head.prev = None
            else:
                self.tail = None

        # If deleting tail
        elif temp == self.tail:
            self.tail = temp.prev
            self.tail.next = None
```

```
        # Middle node
        else:
            temp.prev.next = temp.next
            temp.next.prev = temp.prev

        print("Node deleted successfully")


dll = DoublyLinkedList()

while True:
    print("\n1.APPEND  2.DISPLAY  3.SEARCH  4.DELETE  5.EXIT")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter element to insert: "))
        dll.append(Node(data))

    elif choice == 2:
        dll.display()

    elif choice == 3:
        key = int(input("Enter key to search: "))
        dll.search(key)

    elif choice == 4:
        key = int(input("Enter key to delete: "))
        dll.remove(key)

    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

b. Implement CDLL

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class CircularDoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def display(self):
        if self.head is None:
            print("Linked list is empty")
            return

        temp = self.head
        while True:
            print(temp.data, end=" <-> ")
            temp = temp.next
            if temp == self.head:
                break
        print()

    def append(self, newnode):
        if self.head is None:
            self.head = newnode
            self.tail = newnode
            newnode.next = newnode
            newnode.prev = newnode
        else:
            newnode.prev = self.tail
```

```
        newnode.next = self.head
        self.tail.next = newnode
        self.head.prev = newnode
        self.tail = newnode

    def remove(self, key):
        if self.head is None:
            print("Linked list is empty")
            return

        temp = self.head
        while True:
            if temp.data == key:

                # only one node
                if self.head == self.tail:
                    self.head = None
                    self.tail = None

                # deleting head
                elif temp == self.head:
                    self.head = temp.next
                    self.head.prev = self.tail
                    self.tail.next = self.head

                # deleting tail
                elif temp == self.tail:
                    self.tail = temp.prev
                    self.tail.next = self.head
                    self.head.prev = self.tail

                # deleting middle node
                else:
                    temp.prev.next = temp.next
```

```
                temp.next.prev = temp.prev

            print("Node deleted")
            return

        temp = temp.next
        if temp == self.head:
            print("Key not found")
            return


cdll = CircularDoublyLinkedList()

while True:
    print("\n1.APPEND \t 2.DISPLAY \t 3.DELETE \t 4.EXIT")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter element to insert: "))
        cdll.append(Node(data))

    elif choice == 2:
        cdll.display()

    elif choice == 3:
        key = int(input("Enter key to delete: "))
        cdll.remove(key)

    elif choice == 4:
        break

    else:
        print("Invalid choice")
```

WEEK 8

1. Implement Stack Data Structure.

```
class stack:
    def __init__(self):
        self.stack = []

    def length(self):
        return len(self.stack)

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if len(self.stack) == 0:
            return "Stack is empty"
        else:
            return self.stack.pop()

    def display(self):
        if len(self.stack) == 0:
            return "Stack is empty"
        else:
            return self.stack

s = stack()
while True:
    choice = int(input("\n1.PUSH \t 2.POP \t 3.DISPLAY \t 4.LENGTH \t 5.EXIT\n"))
    if choice == 1:
        item = input("Enter element to push into stack: ")
        s.push(item)

    elif choice == 2:
        print("Popped element is:", s.pop())
```

```
elif choice == 3:
    print("Elements in stack:", s.display())

elif choice == 4:
    print("Number of elements in stack:", s.length())

elif choice == 5:
    print("Exiting program...")
    break

else:
    print("Invalid Choice! Please select between 1 to 5.")
```

2. Implement bracket matching using stack.

```
class bracket_matching:

    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if len(self.stack) != 0:
            return self.stack.pop()
        else:
            return None

    def bm(self, expr):
        for ch in expr:
            if ch in ('{', '[', '('):
                self.push(ch)
            elif ch in ('}', ']', ')'):
```

```
            last = self.pop()

            if last is None:
                return False

            if last == '{' and ch == '}':
                continue
            elif last == '[' and ch == ']':
                continue
            elif last == '(' and ch == ')':
                continue
            else:
                return False

        if len(self.stack) > 0:
            return False
        else:
            return True


b = bracket_matching()
expr = str(input("Enter expression: "))
result = b.bm(expr)
if result == True:
    print("Given expression is balanced")
else:
    print("Given expression is not balanced")
```

WEEK 9

9a. Program to demonstrate recursive operations (factorial/ Fibonacci)

```
def fib(n):
  if n == 0 or n==1:
     return n
  else:
     return(fib(n-1) +fib(n-2))

def fact(n):
  if n == 0 or n==1:
     return 1
  else:
     return n*fact(n-1)

n=int(input("Enter value for n"))
print("Fibonacci series of",n)
for i in range(n+1):
    print(fib(i),end=',')

print("\n Factorial of",n)
print(fact(n))
```

9b. Implement solution for Towers of Hanoi.

```
count=0
def toh(n , source, destination, temp):
    if n==1:
        print ("Move disk 1 from source",source,"to ",destination)
        global count
        count=count+1
        return
    toh(n-1, source, temp, destination)
    print ("Move disk",n,"from source",source,"to ",destination)
    count=count+1
```

```
    toh(n-1, temp, destination, source)

n = int(input("Enter number of disks"))
toh(n,'A','B','C')
print("total number of moves",count)
```

WEEK 10

1. Implement Queue.

```
class Queue:
    def __init__(self):
        self.queue = []   # List to store queue elements

    def isempty(self):
        return len(self.queue) == 0

    def length(self):
        print("Number of elements in the queue:", len(self.queue))

    def enqueue(self, item):
        self.queue.append(item)
        print("Contents of the queue:", self.queue)

    def dequeue(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print("Dequeued element is:", self.queue.pop(0))  # Removes first element
            print("Contents of the queue:", self.queue)


q = Queue()
```

```
while True:
    choice = int(input("\n1.ENQUEUE \t 2.DEQUEUE \t 3.LENGTH \t 4.EMPTY \t 5.EXIT\nEnter your choice: "))

    if choice == 1:
        element = int(input("Enter element to enqueue into queue: "))
        q.enqueue(element)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.length()

    elif choice == 4:
        print("Queue is empty:", q.isempty())

    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice!")
```

2. Implement priority queue

```
class que:
    def __init__(self):
        self.queue = []

    def isempty(self):
        return len(self.queue) == 0
```

```
    def length(self):
        print("Number of elements in the queue:", len(self.queue))

    def enqueue(self, item):
        self.queue.append(item)
        print("Element inserted successfully.")

    def dequeue(self):
        if self.isempty():
            print("Queue is empty")
        else:
            priority = 0
            for i in range(len(self.queue)):
                if self.queue[i] < self.queue[priority]:
                    priority = i
            print("Dequeued element is:", self.queue.pop(priority))

    def display(self):
        if self.isempty():
            print("Queue is empty")
        else:
            print("Queue elements are:", self.queue)

q = que()
while True:
    choice = int(input("\n1.ENQUEUE \t 2.DEQUEUE \t 3.LENGTH \t 4.EMPTY \t 5.DISPLAY \t 6.EXIT\nEnter your choice: "))

    if choice == 1:
        element = int(input("Enter element to enqueue into queue: "))
        q.enqueue(element)

    elif choice == 2:
        q.dequeue()
```

```
    elif choice == 3:
        q.length()

    elif choice == 4:
        print("Queue is empty:", q.isempty())

    elif choice == 5:
        q.display()

    elif choice == 6:
        break

    else:
        print("Invalid choice!")
```

WEEK 11

1. Implement Binary search tree and its operations using list.

```
class node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class bst:
    def __init__(self):
        self.root = None

    def insert(self, newnode):
        if self.root is None:
            self.root = newnode
            return
```

```
        else:
            temp = self.root
            while True:
                if newnode.data < temp.data:
                    if temp.left is None:
                        temp.left = newnode
                        return
                    else:
                        temp = temp.left
                else:
                    if temp.right is None:
                        temp.right = newnode
                        return
                    else:
                        temp = temp.right

    def search(self, key):
        temp = self.root
        if temp is None:
            return "Tree is empty"

        while True:
            if temp is None:
                return "Element not found"

            if key == temp.data:
                return "Element found"
            elif key < temp.data:
                temp = temp.left
            else:
                temp = temp.right

    def inorder(self, root):
        if root is None:
```

```
            return
        self.inorder(root.left)
        print(root.data)
        self.inorder(root.right)


b = bst()

while True:
    choice = int(input("\n1. INSERT \t 2. SEARCH \t 3. DISPLAY \t 4. EXIT\nEnter Your Choice: "))

    if choice == 1:
        item = int(input("Enter an element: "))
        b.insert(node(item))

    elif choice == 2:
        key = int(input("Enter key to search: "))
        print(b.search(key))

    elif choice == 3:
        print("Inorder Traversal:")
        b.inorder(b.root)

    elif choice == 4:
        break

    else:
        print("Invalid choice!")
```

WEEK 12

1. Implementations of BFS.

```
from collections import deque


class node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class bst:
    def __init__(self):
        self.root = None

    def insert(self, newnode):
        if self.root is None:
            self.root = newnode
            return
        else:
            temp = self.root
            while True:
                if newnode.data < temp.data:
                    if temp.left is None:
                        temp.left = newnode
                        return
                    else:
                        temp = temp.left
                else:
                    if temp.right is None:
                        temp.right = newnode
                        return
                    else:
                        temp = temp.right
```

```
    def bfs(self):
        nodes = []
        if self.root is None:
            print("Tree is Empty")
            return

        queue = deque([self.root])

        while len(queue) > 0:
            temp = queue.popleft()
            nodes.append(temp.data)

            if temp.left:
                queue.append(temp.left)
            if temp.right:
                queue.append(temp.right)

        return nodes


# Main Program
b = bst()

while True:
    choice = int(input("\n1. INSERT \t 2. BFS Traversal \t 3. EXIT\nEnter your choice: "))

    if choice == 1:
        item = int(input("Enter element to insert into Tree: "))
        b.insert(node(item))

    elif choice == 2:
        print("BFS traversal is:", b.bfs())
```

```
    elif choice == 3:
        print("Exiting...")
        break

    else:
        print("Invalid choice! Try again.")
```

2. Implementation of DFS.

```
class node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class bst:
    def __init__(self):
        self.root = None

    def insert(self, newnode):
        if self.root is None:
            self.root = newnode
            return
        else:
            temp = self.root
            while True:
                if newnode.data <= temp.data:
                    if temp.left is None:
                        temp.left = newnode
                        return
                    else:
                        temp = temp.left
                else:
                    if temp.right is None:
```

```
                temp.right = newnode
                return
            else:
                temp = temp.right

    def preorder(self, root):
        if root is None:
            return
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

    def inorder(self, root):
        if root is None:
            return
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)

    def postorder(self, root):
        if root is None:
            return
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")


# Main Program
b = bst()

while True:
    choice = int(input(
        "\n1. INSERT\n2. DFS PREORDER\n3. DFS INORDER\n4. DFS POSTORDER\n5. EXIT\nEnter
your choice: "
```

```
))

if choice == 1:
    item = int(input("Enter element to insert into Tree: "))
    b.insert(node(item))

elif choice == 2:
    print("Preorder traversal is: ", end="")
    b.preorder(b.root)
    print()

elif choice == 3:
    print("Inorder traversal is: ", end="")
    b.inorder(b.root)
    print()

elif choice == 4:
    print("Postorder traversal is: ", end="")
    b.postorder(b.root)
    print()

elif choice == 5:
    print("Exiting...")
    break

else:
    print("Invalid choice!")
```

WEEK 13

1. Implement Hash functions.

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    # Hash Function
    def hash_function(self, key):
        return key % self.size

    # Insert using Linear Probing
    def insert(self, key):
        index = self.hash_function(key)

        if self.table[index] is None:
            self.table[index] = key
        else:
            print("Collision occurred at index", index)
            original_index = index
            index = (index + 1) % self.size

            while index != original_index:
                if self.table[index] is None:
                    self.table[index] = key
                    return
                index = (index + 1) % self.size

            print("Hash Table is Full")

    # Display Hash Table
    def display(self):
        print("\nHash Table:")
```

```
        for i in range(self.size):
            print("Index", i, ":", self.table[i])


# Main Program
size = int(input("Enter size of Hash Table: "))
h = HashTable(size)

while True:
    print("\n1. Insert\n2. Display\n3. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter key to insert: "))
        h.insert(key)

    elif choice == 2:
        h.display()

    elif choice == 3:
        break

    else:
        print("Invalid choice!")
```